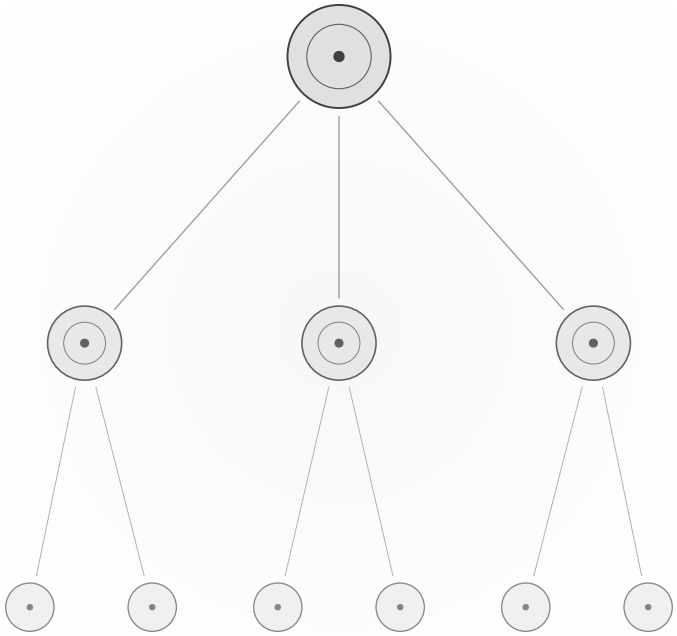

SHA-256 6E:0F:59:4D:9A:3D:6D:65:FE:E8:F7:EE:F9:95:DB:05
88:17:88:AB:3D:E8:6A:48:93:FF:AD:00:B9:F9:A6:25

pki-tools

A Practical Guide to Private PKI

From "Why Bother" to "Here's How"

Chiradip Mandal

February 2025

Published by
Space-RF.ORG
Academy Of Distributed Computing

Contents

I Why 1

1 The Problem with Certificates 2
- 1.1 The One-Certificate Problem . . . 2
- 1.2 The Real Problem: Hierarchies . . . 2
- 1.3 Where It Gets Worse . . . 4
- 1.4 The Honest Value Proposition . . . 5

2 What the Suite Does 6
- 2.1 What It Is . . . 6
- 2.2 What It Is Not . . . 6

3 When to Use a Private PKI 8
- 3.1 Mutual TLS (mTLS) . . . 8
- 3.2 Air-Gapped or Restricted Networks . . . 8
- 3.3 Organizational Control . . . 8
- 3.4 Compliance . . . 8

4 The Cost of Not Having Tools 9
- 4.1 The Quiet Accumulation . . . 9
- 4.2 The 2 a.m. Expiry . . . 9
- 4.3 The Departed Engineer . . . 9
- 4.4 The Revocation You Never Built . . . 9
- 4.5 The Policy Drift . . . 10

5 Architecture Decisions 11
- 5.1 Seven Binaries, One Library . . . 11
- 5.2 No Configuration Files . . . 11
- 5.3 Defaults That Work . . . 11
- 5.4 Flat File Database . . . 12
- 5.5 No Daemon, No State Between Invocations . . . 12

II Certificates 13

6 Installation 14
- 6.1 Requirements . . . 14
- 6.2 Building . . . 14
- 6.3 Testing . . . 14

7 Quick Start 16

8 Global Options 17
- 8.1 `--env-prefix` . . . 17
- 8.2 `--output-format` . . . 17

9 The Passphrase **18**

10 Key Algorithms **19**

11 Certificate Profiles **20**

12 Subject Alternative Names **21**

13 Building a PKI Hierarchy **22**
13.1 Example: A Complete Hierarchy 22

14 PKCS#12 Bundles **24**

15 The CSR Workflow **25**
15.1 Step 1: Requester Creates a CSR 25
15.2 Step 2: CA Operator Signs the CSR 25
15.3 Step 3: Requester Uses the Certificate 25

16 Inspection and Monitoring **26**
16.1 Inspecting Certificates 26
16.2 Checking Expiration 26

17 Revocation and CRLs **28**
17.1 Revoking a Certificate 28
17.2 Generating a CRL 28
17.3 Important Note 28

18 Renewal **29**

19 The Serial Database **30**
19.1 Listing Certificates 30

20 CA Constraints **31**
20.1 Path Length (`--pathlen`) 31
20.2 Name Constraints 31

21 Private Key Encryption **32**

III The Rest of the Job **33**

22 Certificate Monitoring **34**
22.1 Scanning Files 34
22.2 Scanning from a Config File 34
22.3 Probing Live TLS Endpoints 34
22.4 Continuous Monitoring 35
22.5 Exit Codes 35

23 TLS Inspection **36**
23.1 Basic Probe 36
23.2 Custom Trust Store 36
23.3 SNI Override 36

23.4 Mutual TLS Testing . . . 36
23.5 Batch Scanning . . . 37
23.6 JSON Output . . . 37

24 Format Conversion **38**
24.1 Converting Formats . . . 38
24.2 Splitting a PKCS#12 . . . 38
24.3 Merging PEM Files . . . 38
24.4 Inspecting Any Format . . . 39
24.5 Format Auto-Detection . . . 39

25 Policy Validation **40**
25.1 Writing a Policy . . . 40
25.2 Linting a Certificate . . . 40
25.3 Linting a Directory . . . 41
25.4 Displaying a Policy . . . 41
25.5 Practical Use . . . 41

26 Digital Signatures **42**
26.1 Signing a File . . . 42
26.2 Verifying a Signature . . . 42
26.3 Tamper Detection . . . 42
26.4 Hash Algorithms . . . 43

27 Hybrid Encryption **44**
27.1 Encrypting . . . 44
27.2 Decrypting . . . 44
27.3 Why CMS, Not GPG . . . 44
27.4 Binary Files . . . 45

28 JSON Output for Automation **46**
28.1 CI/CD Pipelines . . . 46
28.2 Inventory Management . . . 46
28.3 Audit Logging . . . 46

29 Custom Scripts **47**

30 Reference Card **48**
30.1 `pki-tool` . . . 48
30.2 `cert-monitor` . . . 48
30.3 `tls-probe` . . . 49
30.4 `keystore-tool` . . . 49
30.5 `cert-lint` . . . 49
30.6 `sign-tool` . . . 49
30.7 `secret-tool` . . . 49

31 Common Mistakes and Troubleshooting **51**
31.1 “Cannot read CA key” . . . 51
31.2 Forgot `--db` During Issuance . . . 51
31.3 Chain Verification Fails . . . 51
31.4 Wrong Profile After Issuance . . . 52

31.5 PKCS#12 Import Fails on Another System 52
31.6 Ed25519 Not Recognized by Client 52
31.7 `sign-tool verify` Fails with "unsuitable certificate purpose" 52
31.8 `cert-lint` Reports Failures on Certificates You Just Issued 52

32 Putting It All Together **54**
32.1 Initial Setup (Once) 54
32.2 Issuing Certificates (As Needed) 54
32.3 Monitoring (Weekly Cron) 55
32.4 Renewal (When Warned) 55
32.5 Signing a Release (As Needed) 56
32.6 Encrypting Secrets (As Needed) 56
32.7 Quarterly Audit 56

IV Getting pki-tools **57**

33 Download the Trial **58**
33.1 Download 58
33.2 Install 58
33.3 Trial Behavior 58
33.4 Restrictions 59

34 Buy a Full License **60**
34.1 License Models 60
34.2 Pricing 60
34.3 Buy Online 60
34.4 Volume Discounts 61
34.5 Contact 61

Part I

Why

The Problem with Certificates

Let's be direct. You can do everything `pki-tools` does with raw OpenSSL commands. OpenSSL is installed on virtually every Linux box. It is free. It is well-documented. So why would you use anything else?

Because OpenSSL is a *cryptographic library* with a command-line interface bolted on. It gives you total control over every field in every ASN.1 structure. That is exactly what you need when you are implementing a TLS library. It is not what you need when you are trying to issue a server certificate at 4 p.m. on a Friday.

Here is what that distinction looks like in practice.

1.1 The One-Certificate Problem

Issuing a single TLS certificate with OpenSSL:

```
openssl ecparam -genkey -name prime256v1 -noout -out key.pem
openssl req -new -key key.pem -out req.pem -subj "/CN=my-server"
openssl x509 -req -in req.pem -CA ca-cert.pem -CAkey ca-key.pem \
    -CAcreateserial -out cert.pem -days 365 \
    -extfile <(printf "basicConstraints=critical,CA:FALSE\n\
keyUsage=critical,digitalSignature,keyEncipherment\n\
extKeyUsage=serverAuth,clientAuth\n\
subjectKeyIdentifier=hash\n\
authorityKeyIdentifier=keyid:always")
```

Seven lines. The `-extfile` with process substitution is fragile—a missing newline breaks it silently. Forget `basicConstraints` and the certificate looks valid but chain verification fails in some TLS libraries. Forget `subjectKeyIdentifier` and you have a technically non-compliant X.509v3 certificate that works today and might not tomorrow.

The same operation with `pki-tool`:

```
pki-tool issue-cert --ca ./my-ca --cn "my-server" --out ./server
```

One line. Extensions are correct. Serial is 128-bit random. Key permissions are 0600. There is nothing to forget.

1.2 The Real Problem: Hierarchies

One certificate is manageable. A real deployment is not. Here is a minimal PKI hierarchy with OpenSSL: a root CA, one intermediate CA, and one end-entity certificate.

```
# Root CA key and self-signed cert
openssl ecparam -genkey -name prime256v1 -noout -out root-key.pem
openssl req -new -x509 -key root-key.pem -out root-cert.pem \
```

```
    -days 3650 -subj "/CN=My Root CA" \
    -addext "basicConstraints=critical,CA:TRUE,pathlen:1" \
    -addext "keyUsage=critical,keyCertSign,cRLSign,digitalSignature" \
    -addext "subjectKeyIdentifier=hash"
chmod 600 root-key.pem

# Intermediate CA key and CSR
openssl ecparam -genkey -name prime256v1 -noout -out sub-key.pem
openssl req -new -key sub-key.pem -out sub.csr \
    -subj "/CN=My Server CA"

# Sign the intermediate with the root (needs an extfile)
cat > sub-ext.cnf <<'EXTEOF'
basicConstraints = critical,CA:TRUE,pathlen:0
keyUsage = critical,keyCertSign,cRLSign,digitalSignature
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid:always
EXTEOF
openssl x509 -req -in sub.csr -CA root-cert.pem \
    -CAkey root-key.pem -CAcreateserial -out sub-cert.pem \
    -days 1825 -extfile sub-ext.cnf
chmod 600 sub-key.pem

# Build chain file
cat sub-cert.pem root-cert.pem > chain.pem

# End-entity key and CSR
openssl ecparam -genkey -name prime256v1 -noout -out srv-key.pem
openssl req -new -key srv-key.pem -out srv.csr \
    -subj "/CN=api.example.com"

# Sign with the intermediate (different extfile!)
cat > srv-ext.cnf <<'EXTEOF'
basicConstraints = critical,CA:FALSE
keyUsage = critical,digitalSignature,keyEncipherment
extKeyUsage = serverAuth,clientAuth
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid:always
subjectAltName = DNS:api.example.com
EXTEOF
openssl x509 -req -in srv.csr -CA sub-cert.pem \
    -CAkey sub-key.pem -CAcreateserial -out srv-cert.pem \
    -days 365 -extfile srv-ext.cnf
chmod 600 srv-key.pem

# Bundle into PKCS#12
openssl pkcs12 -export -in srv-cert.pem -inkey srv-key.pem \
    -certfile chain.pem -out server.p12 \
    -passout env:PKI_KEYSTORE_PASS -name "api.example.com"

# Clean up temp files
rm -f sub.csr srv.csr sub-ext.cnf srv-ext.cnf \
    root-cert.srl sub-cert.srl
```

That is 45 lines for one root CA, one intermediate, and one end-entity certificate. You

wrote two separate extension configuration files. You manually built the chain file. You manually set file permissions. You cleaned up serial files that OpenSSL left behind.

The same hierarchy with `pki-tool`:

```
pki-tool init-ca --cn "My Root CA" --days 3650 \
    --pathlen 1 --out ./root

pki-tool create-sub-ca --parent ./root \
    --cn "My Server CA" --days 1825 --out ./sub-ca

pki-tool issue-cert --ca ./sub-ca --cn "api.example.com" \
    --days 365 --san "DNS:api.example.com" --out ./srv

pki-tool create-p12 --cert ./srv/cert.pem \
    --key ./srv/key.pem --chain ./sub-ca/chain.pem \
    --out ./server.p12
```

Four commands. No temporary files. No extension configs. The chain file is built automatically. Key permissions are set automatically. Serial numbers are random. Every certificate has the correct extensions for its role.

1.3 Where It Gets Worse

The OpenSSL approach does not just start ugly—it degrades over time.

Revocation. OpenSSL's CRL generation requires an `openssl.cnf` configuration file with a `[ca]` section pointing to a serial file, an index file, and a CRL number file—none of which exist unless you create them by hand, in the right format, with the right initial values. Most teams skip revocation entirely because the setup cost is too high. With `pki-tool`:

```
pki-tool revoke --ca ./sub-ca --cert ./srv/cert.pem
pki-tool generate-crl --ca ./sub-ca --days 7
```

Two commands. The serial database was populated when you used `--db` during issuance.

Renewal. OpenSSL has no concept of renewal. You repeat the entire issuance sequence, hoping you match the same CN and SANs as the original. With `pki-tool`:

```
pki-tool renew --cert ./srv/cert.pem --ca ./sub-ca \
    --days 365 --out ./srv-v2
```

One command. The CN and SANs are extracted from the original certificate automatically.

Tracking. OpenSSL does not track what it issues. After six months, you have PEM files scattered across directories and no record of which CA signed what, when things expire, or what has been revoked. `pki-tool` maintains a pipe-delimited serial database—one `grep` tells you the state of every certificate.

1.4 The Honest Value Proposition

`pki-tools` is a suite of C programs. They call the same OpenSSL library functions you would call. They do not add cryptographic capabilities—they remove the operational burden.

The value is not in what they compute. The value is in what they prevent: misconfigured extensions, forgotten permissions, lost track of expiry, absent revocation infrastructure, policy drift, undetected TLS misconfigurations, and the institutional fragility of a PKI that only one person understands.

If your team has a senior engineer who enjoys maintaining 200-line shell scripts full of OpenSSL commands, and that engineer will never leave or be sick on the day a certificate expires—you do not need these tools. Everyone else does.

What the Suite Does

Before we get into the details, it is worth stating clearly what this project *is* and what it *is not*.

2.1 What It Is

`pki-tools` is a suite of seven command-line programs with no dependencies beyond OpenSSL 3.x. They share a static library of common utilities and a consistent interface. Together they cover the X.509 lifecycle from key generation to certificate expiry monitoring, with the practical additions of TLS inspection, format conversion, policy enforcement, digital signatures, and hybrid encryption.

Binary	**Purpose**
`pki-tool`	Certificate authority management: CA init, sub-CAs, cert issuance, CSRs, CRLs, revocation, renewal, inspection
`cert-monitor`	Certificate expiry scanning: files, directories, serial databases, live TLS endpoints, continuous watch mode
`tls-probe`	TLS handshake inspection: protocol version, cipher suite, certificate chain, verification result, mTLS testing
`keystore-tool`	Format conversion: PEM, DER, PKCS#12 with split, merge, convert, and auto-detection
`cert-lint`	Policy validation: enforce key sizes, validity periods, extensions, algorithms, SAN requirements
`sign-tool`	Detached CMS/PKCS#7 signatures: sign files and verify signatures with configurable hash algorithms
`secret-tool`	Hybrid encryption via CMS EnvelopedData: multi-recipient encrypt and decrypt

Table 2.1: The seven tools

Every tool supports `--output-format json|text` and `--env-prefix PREFIX` for passphrase resolution via environment variables.

2.2 What It Is Not

It is not a certificate authority in the ACME/Let's Encrypt sense. None of these programs run as daemons (except `cert-monitor watch`, which is deliberately simple). There is no web interface. There is no certificate distribution or deployment system.

These are command-line tools that read files, do their work, and write files. What you do with those files is your problem.

This distinction matters. If you need automated certificate rotation for 500 microservices, you need something like cert-manager or Vault. If you need to build a private PKI hierarchy, issue certificates, monitor their expiry, validate them against policy, and sign your release artifacts—that is what these tools do.

When to Use a Private PKI

Public certificate authorities (Let's Encrypt, DigiCert, etc.) are the right choice for public-facing websites. They are trusted by browsers, they are free or cheap, and ACME automation handles the renewal. There is no reason to build your own CA for a public website.

Private PKI makes sense in a different set of situations:

3.1 Mutual TLS (mTLS)

When both sides of a connection need to prove their identity—not just the server—you need client certificates. Public CAs will not issue client certificates for your internal services. You need your own CA.

This is common in microservice architectures, IoT device fleets, and any system where the server needs to verify that the connecting client is authorized, not just encrypted.

3.2 Air-Gapped or Restricted Networks

If your systems cannot reach the internet (defense, industrial control, maritime), ACME is not an option. You need certificates generated offline and distributed through secure channels.

3.3 Organizational Control

With your own CA, you control the trust boundary. A certificate issued by your CA is valid only within your infrastructure. If a key is compromised, you revoke it and regenerate—without waiting for a third party.

You also control the certificate contents precisely: the algorithms, the extensions, the validity periods, the name constraints. No one else's policy decisions affect your infrastructure.

3.4 Compliance

Certain compliance frameworks (CMMC, FedRAMP, PCI-DSS) require or strongly encourage private PKI for internal communications. Having tools that produce auditable output (serial database, CRLs, JSON-format inspection, policy validation reports) helps with the evidence-gathering side of compliance.

The Cost of Not Having Tools

Every company that manages its own certificates without tooling ends up in the same place. Not immediately—it takes six months to a year. But the destination is always the same.

4.1 The Quiet Accumulation

You start with one certificate. Someone writes a shell script. It works. The next certificate is a copy of that script with a different CN. Also works. By month six you have fourteen certificates across three servers, each created by a slightly different copy of a slightly different script. Some use RSA, some use EC. Some set `extKeyUsage`, some don't. One has `CA:TRUE` on an end-entity cert because someone copied the wrong template. It all works until it doesn't.

4.2 The 2 a.m. Expiry

Nobody wrote down when the certificates expire. The monitoring system's own certificate expires first, so you lose visibility into the problem before you can detect it. You find out when a customer reports that your API returns TLS errors.

`cert-monitor` solves this: point it at a directory of PEM files, a serial database, or a list of live TLS endpoints, and it tells you what is expiring. Run it in watch mode and it checks continuously. But monitoring only works on certificates you can find, and you can only find certificates that were tracked when they were issued. Retrofitting tracking onto an existing pile of PEM files is painful. Starting with tracking from day one costs nothing—it's a single `--db` flag.

4.3 The Departed Engineer

The person who set up the PKI leaves. Their shell history on a decommissioned laptop is the only record of how certificates were generated. The next person reads the surviving PEM files, reverse-engineers the hierarchy from issuer fields, and starts over. This happens more often than anyone admits.

A script that calls `pki-tool` is documentation. The command names are the procedure. A new engineer reads `pki-tool issue-cert --ca ./server-ca --cn "api" --profile tls-server` and knows exactly what happened. They read `openssl x509 -req -in req.pem -extfile ext.cnf` and know nothing until they also find `ext.cnf`, which was deleted after the script ran.

4.4 The Revocation You Never Built

A server is decommissioned. Its private key sits on a disk image in cold storage. Without a CRL, that key can still authenticate against your infrastructure. The fix is to revoke the

certificate and publish a CRL. But OpenSSL's CRL workflow requires a configuration file, an index database, a serial counter, and a CRL number file—infrastructure you never set up because it wasn't needed at the time and it's a hassle to build retroactively.

With `pki-tool`, revocation is two commands. But only if you used `--db` when you issued the certificate. This is why the tools exist: not to make the easy case easier (they do that too), but to make the hard cases possible at all.

4.5 The Policy Drift

You wrote a wiki page that says "all certificates must use EC P-256 and be valid for no more than one year." Six months later, someone issues an RSA-2048 cert valid for three years because they copied an old script. Nobody notices until an auditor does.

`cert-lint` turns your policy from a wiki page into an executable check. Write the rules in a policy file. Run the linter against every certificate. The tool does not care about good intentions; it checks what actually exists.

Architecture Decisions

A few design choices are worth explaining, because they affect when these tools are and are not appropriate.

5.1 Seven Binaries, One Library

The suite is structured as seven standalone executables linked against a shared static library (`libpki-shared.a`). The library contains common utilities: PEM I/O, JSON formatting, passphrase resolution, certificate helpers.

Each tool is self-contained. You can deploy only the binaries you need. A server that only monitors certificate expiry needs only `cert-monitor`. A build system that signs release artifacts needs only `sign-tool` and a signing certificate.

5.2 No Configuration Files

Everything is specified on the command line or through environment variables. There are no configuration files, no INI templates, no YAML. This is deliberate. Configuration files create a second source of truth that can drift from reality. Command-line invocations are explicit and scriptable.

The one exception is `cert-lint`'s policy file, which is a configuration file by nature—it defines the rules. But it is not an application config; it is an input to the linter, like a source file is an input to a compiler.

The trade-off is that complex operations require long command lines. For recurring tasks, the answer is a shell script—like the `custom/generate-roip-certs.sh` example included with the project.

5.3 Defaults That Work

The default key algorithm is EC P-256. The default profile is `tls-mutual` (serverAuth + clientAuth). The default CA validity is 10 years. The default end-entity validity is 1 year. Serial numbers are 128-bit random.

These defaults are not arbitrary. P-256 is the most widely supported elliptic curve. Mutual TLS is the most permissive profile (a cert that can do both server and client auth is strictly more useful than one that can do only one). Random serials avoid the hazards of sequential serial number assignment.

If any of these defaults are wrong for your use case, every one of them can be overridden.

5.4 Flat File Database

The serial database is a pipe-delimited text file. This is the simplest possible design. It can be read with `cat`, searched with `grep`, backed up with `cp`, and version-controlled with `git`.

The trade-off is that it does not scale to millions of certificates, and concurrent writes from multiple processes could corrupt it. For the use case these tools target (tens to thousands of certificates, managed by scripts or a small team), a text file is perfectly adequate.

5.5 No Daemon, No State Between Invocations

Each invocation reads files, does its work, and writes files. There is no lock file, no PID file, no socket, no shared memory. This makes the tools trivially parallelizable (as long as you don't write to the same serial database concurrently) and trivially debuggable (every input and output is a file you can inspect).

The only exception is `cert-monitor watch`, which loops and sleeps. It is still stateless between iterations—each scan starts fresh.

Part II

Certificates

Installation

6.1 Requirements

- A C11 compiler (GCC or Clang)
- CMake 3.16 or later
- OpenSSL 3.x development libraries

On Debian/Ubuntu:

```
sudo apt install build-essential cmake libssl-dev
```

On Fedora/RHEL:

```
sudo dnf install gcc cmake openssl-devel
```

On macOS (with Homebrew):

```
brew install cmake openssl@3
```

6.2 Building

```
git clone <repository-url> pki-tools
cd pki-tools
mkdir build && cd build
cmake .. && make
```

The build produces a static library (`libpki-shared.a`) and seven executables: `pki-tool`, `cert-monitor`, `tls-probe`, `keystore-tool`, `cert-lint`, `sign-tool`, and `secret-tool`.

6.3 Testing

```
cd build
ctest --output-on-failure
```

Seven test suites exercise approximately 210 assertions across every command, every key algorithm, every tool, and both output formats.

Test	**Assertions**	**Coverage**
`pki`	∼100	CA lifecycle, all key algorithms, SANs, profiles, CSR workflow, serial DB, revocation, CRL, renewal, constraints, JSON
`cert-monitor`	∼15	File/directory/serial-DB scan, config file, JSON output, near-expiry detection
`tls-probe`	∼15	TLS connect via local `openssl s_server`, chain verification, JSON, batch scan
`keystore-tool`	∼20	P12↔PEM split/merge, PEM↔DER roundtrip, format auto-detection, info
`cert-lint`	∼25	Passing/failing certs, all policy rules, directory lint, CA-specific rules, JSON
`sign-tool`	∼20	Sign/verify roundtrip, tamper detection, wrong CA rejection, SHA-384/512, PEM format
`secret-tool`	∼15	Encrypt/decrypt roundtrip, wrong key rejection, multi-recipient, binary files, PEM format

Table 6.1: Test suites

Quick Start

Here is the minimum to get a working PKI hierarchy with end-entity certificates, validated against policy and monitored for expiry.

```
export PKI_KEYSTORE_PASS="changeme"

# Root CA
pki-tool init-ca --cn "My Root CA" --days 3650 --out ./root-ca --db

# Intermediate CA
pki-tool create-sub-ca --parent ./root-ca \
    --cn "My Server CA" --days 1825 --out ./server-ca --db

# Server certificate
pki-tool issue-cert --ca ./server-ca --cn "api.example.com" \
    --days 365 --san "DNS:api.example.com" \
    --profile tls-server --out ./api-cert --db

# PKCS#12 bundle (for deployment)
pki-tool create-p12 --cert ./api-cert/cert.pem \
    --key ./api-cert/key.pem \
    --chain ./server-ca/chain.pem --out ./api.p12

# Scan for expiry
cert-monitor scan --cert ./api-cert/cert.pem --warn-days 90
```

That is five commands. You now have a root CA, an intermediate CA, a server certificate with a PKCS#12 bundle, and you know when it expires.

Global Options

Every tool in the suite accepts two global options, which must appear *before* the command name.

8.1 `--env-prefix`

Controls which environment variable is consulted for passphrases.

```
# Default: reads PKI_KEYSTORE_PASS
pki-tool create-p12 ...

# Override: reads MYAPP_KEYSTORE_PASS
pki-tool --env-prefix MYAPP create-p12 ...
```

This exists so that multiple projects can coexist on the same machine without sharing passphrases. The prefix applies across all seven tools.

8.2 `--output-format`

Switches output between human-readable text and machine-parseable JSON.

```
# Text (default)
pki-tool inspect --cert ./cert.pem

# JSON
pki-tool --output-format json inspect --cert ./cert.pem
cert-monitor --output-format json scan --dir /etc/ssl/certs
tls-probe --output-format json connect --target example.com:443
```

JSON output is useful for scripting, CI/CD pipelines, and feeding into monitoring systems. Every command in every tool respects this flag.

The Passphrase

Certificates themselves are not passphrase-protected. Private keys and PKCS#12 bundles can be. `pki-tools` resolves passphrases through a four-step chain, evaluated in order:

1. `--pass "literal"` — given directly on the command line
2. `--pass-file /path/to/file` — read from the first line of a file
3. `<PREFIX>_KEYSTORE_PASS` environment variable
4. Empty string (no passphrase)

The `--pass` and `--pass-file` flags are available on commands that consume protected files (`create-p12`, `inspect`, `keystore-tool` commands, `secret-tool decrypt`). For key encryption (`--encrypt-key`), the passphrase comes from the environment variable, since `init-ca` and `issue-cert` do not accept `--pass`.

This design means you set `PKI_KEYSTORE_PASS` once in your shell session (or CI secret), and every tool picks it up automatically.

```
export PKI_KEYSTORE_PASS="s3cret"
pki-tool init-ca --cn "My CA" --out ./ca --encrypt-key
# Key encrypted with "s3cret"

pki-tool issue-cert --ca ./ca --cn "server" --out ./srv
# CA key decrypted with "s3cret" automatically

pki-tool create-p12 --cert ./srv/cert.pem --key ./srv/key.pem \
    --out ./srv.p12
# P12 protected with "s3cret"
```

Key Algorithms

The `--algo` flag is accepted by `init-ca`, `create-sub-ca`, `issue-cert`, and `create-csr`.

Value	Type	Bits	Notes
`p256`	EC (NIST P-256)	256	Default. Broadest compatibility.
`p384`	EC (NIST P-384)	384	Higher security margin.
`ed25519`	Edwards curve	256	Fast, deterministic signatures.
`rsa2048`	RSA	2048	Legacy compatibility.
`rsa4096`	RSA	4096	Maximum RSA strength.

Table 10.1: Supported key algorithms

All algorithms use SHA-256 for the signature hash, except Ed25519 which uses its own built-in hash (effectively SHA-512 internally, but this is transparent to the user).

A few practical notes:

P-256 is the right choice for most deployments. It is supported by every TLS library, every HSM, and every compliance framework. Key generation is fast. Signatures are small.

Ed25519 is technically superior (faster, deterministic, no side-channel issues from lazy implementations), but support is not universal. Check that your TLS stack and your clients all support it before committing.

RSA is for backward compatibility with systems that do not support elliptic curves. If you have a choice, prefer EC.

A CA and its issued certificates do not need to use the same algorithm. An Ed25519 root CA can sign P-256 end-entity certificates, and vice versa. The signing algorithm is determined by the CA's key type, not the subject's.

Certificate Profiles

When issuing end-entity certificates, the `--profile` flag selects a predefined set of X.509v3 extensions.

Profile	keyUsage	extKeyUsage
`tls-server`	digitalSignature, keyEncipherment	serverAuth
`tls-client`	digitalSignature	clientAuth
`tls-mutual`	digitalSignature, keyEncipherment	serverAuth, clientAuth
`code-signing`	digitalSignature	codeSigning
`email`	digitalSignature, keyEncipherment	emailProtection

Table 11.1: Certificate profiles

All profiles set `basicConstraints: critical, CA:FALSE`.

The default profile is `tls-mutual`, which is the most permissive: the certificate can act as both a server and a client. If you know a certificate will only ever be used in one role, use the more restrictive profile. This follows the principle of least privilege and makes auditing easier.

```
# Server-only certificate
pki-tool issue-cert --ca ./ca --cn "api.example.com" \
    --profile tls-server --out ./api

# Client-only certificate
pki-tool issue-cert --ca ./ca --cn "agent-42" \
    --profile tls-client --out ./agent

# Code signing certificate
pki-tool issue-cert --ca ./ca --cn "build-system" \
    --profile code-signing --out ./signer
```

Subject Alternative Names

The Common Name (CN) in a certificate subject is technically deprecated for host identification. Modern TLS libraries check Subject Alternative Names (SANs) instead. The `--san` flag lets you specify them.

```
pki-tool issue-cert --ca ./ca --cn "api.example.com" \
    --san "DNS:api.example.com,DNS:api-internal.example.com,IP:10.0.1.5" \
    --out ./api
```

The format is a comma-separated list of typed entries:

- `DNS:hostname` — DNS name
- `IP:address` — IPv4 or IPv6 address
- `email:address` — email address (for S/MIME)

SANs are preserved automatically by the `renew` command. If you renew a certificate that has SANs, the new certificate will have the same SANs unless you explicitly override them with `--san`.

Building a PKI Hierarchy

A well-structured PKI has three levels:

1. **Root CA** — self-signed, long-lived, kept offline
2. **Intermediate CAs** — signed by root, medium-lived, one per purpose
3. **End-entity certificates** — signed by an intermediate, short-lived

The root CA is the trust anchor. Every system that participates in your PKI needs the root CA's certificate (or its PKCS#12 bundle). The root CA's private key should be generated once and then stored securely—ideally offline.

Intermediate CAs exist so that the root key is not exposed to routine operations. If an intermediate CA is compromised, you revoke it and create a new one without changing the trust anchor.

13.1 Example: A Complete Hierarchy

```
export PKI_KEYSTORE_PASS="s3cret"

# Root CA (10 years, RSA-4096 for longevity)
pki-tool init-ca --cn "Acme Root CA" --days 3650 \
    --algo rsa4096 --pathlen 1 --out ./root --db

# Server CA (5 years, P-256 for performance)
pki-tool create-sub-ca --parent ./root --cn "Acme Server CA" \
    --days 1825 --permitted-dns ".acme.com" --out ./server-ca --db

# Client CA (5 years)
pki-tool create-sub-ca --parent ./root --cn "Acme Client CA" \
    --days 1825 --out ./client-ca --db

# Server cert with SANs
pki-tool issue-cert --ca ./server-ca --cn "api.acme.com" \
    --days 365 --profile tls-server \
    --san "DNS:api.acme.com,DNS:api-internal.acme.com" \
    --out ./api-cert --db

# Client cert
pki-tool issue-cert --ca ./client-ca --cn "agent-1" \
    --days 365 --profile tls-client --out ./agent-cert --db
```

A few things to notice:

- The root CA uses `--pathlen 1`, which means it can sign intermediate CAs but those intermediate CAs cannot sign further CAs. This limits the depth of the hierarchy.
- The server CA uses `--permitted-dns ".acme.com"`, which means it can only issue certificates for `*.acme.com` hostnames. If someone compromises the server CA key,

they cannot issue certificates for arbitrary domains.

- The root CA uses RSA-4096 (because it will be around for a decade and we want the largest safety margin), while the intermediate CAs and end-entity certs use the default P-256 (because they are shorter-lived and performance matters more).
- Every command uses `--db` to log issued certificates to the serial database.

PKCS#12 Bundles

PKCS#12 is the standard format for distributing a private key together with its certificate and the CA chain, all in one encrypted file. Most TLS libraries can consume PKCS#12 directly.

```
# Bundle server cert with chain
pki-tool create-p12 --cert ./api-cert/cert.pem \
    --key ./api-cert/key.pem \
    --chain ./server-ca/chain.pem \
    --out ./api.p12

# Trust anchor (root CA only, for clients)
pki-tool create-p12 --cert ./root/ca-cert.pem \
    --key ./root/ca-key.pem \
    --out ./trust.p12
```

The `--chain` flag takes the `chain.pem` file produced by `create-sub-ca`, which contains the intermediate CA cert followed by the root CA cert. This gives the receiving system the full chain it needs to verify the end-entity certificate.

The passphrase protecting the PKCS#12 file comes from the passphrase resolution chain described earlier. The most common pattern is to set `PKI_KEYSTORE_PASS` in the environment and let all commands use it.

For more complex format conversions—splitting a P12 into PEM components, converting PEM to DER, or merging PEM files into a P12 with different options—see the `keystore-tool` chapter.

The CSR Workflow

In some organizations, the entity that needs a certificate is not the same entity that operates the CA. The Certificate Signing Request workflow separates these roles.

15.1 Step 1: Requester Creates a CSR

```
pki-tool create-csr --cn "web.acme.com" \
    --san "DNS:web.acme.com,DNS:www.acme.com" \
    --algo p256 --out ./my-request
```

This produces two files in `./my-request/`:

- `key.pem` — the private key (stays with the requester)
- `request.pem` — the CSR (sent to the CA operator)

The private key never leaves the requester's machine. Only the CSR is sent.

15.2 Step 2: CA Operator Signs the CSR

```
pki-tool sign-csr --ca ./server-ca \
    --csr ./my-request/request.pem \
    --days 365 --profile tls-server \
    --out ./signed --db
```

The CA operator verifies the CSR's signature (automatically—the tool rejects tampered CSRs), applies the appropriate profile, and produces `signed/cert.pem`. The SANs from the CSR are carried forward unless overridden with `--san`.

15.3 Step 3: Requester Uses the Certificate

The requester now has `key.pem` (from step 1) and `cert.pem` (from step 2). They can bundle these into a PKCS#12 file or use them directly with their TLS stack.

Inspection and Monitoring

16.1 Inspecting Certificates

The `inspect` command shows everything about a certificate, PKCS#12 bundle, or CSR.

```
# PEM certificate
pki-tool inspect --cert ./api-cert/cert.pem

# PKCS#12 bundle
pki-tool inspect --p12 ./api.p12 --pass "s3cret"

# CSR
pki-tool inspect --csr ./my-request/request.pem
```

The output includes the serial number, subject and issuer, validity dates, key algorithm and size, signature algorithm, all extensions, and SANs. For PKCS#12 files, it also lists the bundled chain certificates.

For scripting, use JSON output:

```
pki-tool --output-format json inspect --cert ./cert.pem
```

This produces a JSON object with fields like `serial`, `subject`, `not_after`, `key_algorithm`, `is_ca`, and `subject_alt_name`.

16.2 Checking Expiration

```
# Warn if certificate expires within 90 days
pki-tool check-expiry --cert ./api-cert/cert.pem --warn-days 90
```

Exit code 0 means the certificate is valid beyond the warning threshold. Exit code 1 means it is either expiring soon or already expired. This is designed for use in cron jobs and CI pipelines:

```
# In a cron job or CI script
for cert in /etc/pki/certs/*.pem; do
    if ! pki-tool check-expiry --cert "$cert" --warn-days 30; then
        echo "WARNING: $cert is expiring soon"
    fi
done
```

The JSON output includes `days_remaining` as an integer, which is useful for graphing and alerting:

```
pki-tool --output-format json check-expiry --cert ./cert.pem
```

```
{
  "cert": "./cert.pem",
  "subject": "/CN=api.example.com",
  "not_after": "Mar  4 12:00:00 2027 GMT",
  "days_remaining": 364,
  "status": "ok"
}
```

The `check-expiry` command works on one certificate at a time. For scanning directories, serial databases, or live TLS endpoints, see `cert-monitor` in Part III.

Revocation and CRLs

When a private key is compromised or a certificate is no longer needed, you revoke it. Revocation has two steps: marking the certificate as revoked in the database, and generating a Certificate Revocation List (CRL) that clients can check.

17.1 Revoking a Certificate

```
# By certificate file
pki-tool revoke --ca ./server-ca --cert ./api-cert/cert.pem

# By serial number
pki-tool revoke --ca ./server-ca --serial "AB12CD34EF567890..."
```

This updates the certificate's status from `V` (valid) to `R` (revoked) in the CA's `serial.db` file. The certificate must have been issued with `--db` to appear in the database.

17.2 Generating a CRL

```
pki-tool generate-crl --ca ./server-ca --days 7
```

This reads all revoked entries from `serial.db`, creates a signed CRL, and writes it to `server-ca/crl.pem` (or to a path specified with `--out`).

The `--days` flag sets the CRL's validity period. Clients that check CRLs will reject the CRL after this period, so you need to regenerate it periodically. A common pattern is a weekly cron job.

17.3 Important Note

Revocation only works if the clients actually check the CRL. The tool generates the CRL; distributing it and configuring clients to use it is your responsibility.

Renewal

Certificates expire. When they do, you need to issue a new one with the same identity but a new key and new validity period.

```
pki-tool renew --cert ./api-cert/cert.pem \
    --ca ./server-ca --days 365 --out ./api-cert-v2 --db
```

The `renew` command reads the original certificate to extract the Common Name and Subject Alternative Names, generates a fresh key, and issues a new certificate with the same identity information. The SANs are preserved automatically unless you override them with `--san`.

Renewal does *not* automatically revoke the old certificate. If the old certificate is still valid and you want it revoked, do that separately:

```
pki-tool revoke --ca ./server-ca --cert ./api-cert/cert.pem
pki-tool generate-crl --ca ./server-ca --days 7
```

The Serial Database

The serial database is an opt-in feature, activated by the `--db` flag on certificate-issuing commands (`init-ca`, `create-sub-ca`, `issue-cert`, `sign-csr`, `renew`).

The database is a plain text file at `<ca-dir>/serial.db` with one line per issued certificate:

```
SERIAL_HEX|CN|NOT_AFTER|STATUS
```

Where `STATUS` is `V` for valid or `R` for revoked.

19.1 Listing Certificates

```
pki-tool list --ca ./server-ca
```

Produces a table:

```
SERIAL                                CN              NOT AFTER
    STATUS
------                                --              ---------
    ------
0102ABCDEF1234567890ABCDEF123456...  api.acme.com    Mar  4 12:00:00 2027 GMT
    Valid
FEDCBA0987654321FEDCBA0987654321...  agent-1         Mar  4 12:00:00 2027 GMT
    Revoked

Total: 2  Valid: 1  Revoked: 1
```

In JSON mode, this is an array of objects:

```
pki-tool --output-format json list --ca ./server-ca
```

The database is append-only during normal operation (issuing). Revocation rewrites the file in-place to flip the status character. This is simple and correct for single-operator use. If you need concurrent access from multiple processes, serialize through a wrapper script or a lock file.

The serial database is also readable by `cert-monitor`: the `--db` flag to `cert-monitor scan` reads the database and checks every recorded certificate for impending expiry.

CA Constraints

Two mechanisms limit what a CA can do: path length constraints and name constraints.

20.1 Path Length (`--pathlen`)

The `pathlen` value in the `basicConstraints` extension controls how many levels of CA can exist below this one.

```
# This root CA can sign intermediate CAs, but those
# intermediates cannot sign further CAs
pki-tool init-ca --cn "Root CA" --pathlen 1 --out ./root

# This intermediate CA can only sign end-entity certs
pki-tool create-sub-ca --parent ./root --cn "Leaf CA" \
    --pathlen 0 --out ./leaf-ca
```

- `--pathlen 0`: can sign end-entity certs only
- `--pathlen 1`: can sign one level of sub-CAs
- No `--pathlen`: no depth restriction

20.2 Name Constraints

Name constraints restrict the DNS domains for which a CA can issue certificates. They are specified on the CA certificate itself.

```
pki-tool create-sub-ca --parent ./root --cn "Internal CA" \
    --permitted-dns ".internal.acme.com,.corp.acme.com" \
    --excluded-dns ".legacy.acme.com" \
    --out ./internal-ca
```

This intermediate CA can issue certificates only for hostnames ending in `.internal.acme.com` or `.corp.acme.com`, and explicitly cannot issue for `.legacy.acme.com` even though it is a subdomain of what is otherwise permitted.

Name constraints are enforced by the TLS client's certificate verification logic. They are a defense-in-depth measure: even if the intermediate CA's key is stolen, the attacker cannot use it to impersonate hosts outside the permitted domains.

Private Key Encryption

By default, private keys are written unencrypted (but with file permissions set to 0600). The `--encrypt-key` flag encrypts them with AES-256-CBC using the passphrase from the environment variable.

```
export PKI_KEYSTORE_PASS="s3cret"
pki-tool init-ca --cn "My CA" --encrypt-key --out ./ca
```

The resulting `ca-key.pem` will contain an encrypted PEM block. Any subsequent command that needs to read this key (like `issue-cert` or `create-sub-ca`) will automatically try the environment variable passphrase if the unencrypted read fails.

This is useful for keys that are stored at rest on shared filesystems or in version control (though you should think twice before putting private keys in version control, encrypted or not).

Part III

The Rest of the Job

Certificate Monitoring

Issuing certificates is the easy part. Knowing when they expire is the part that wakes you up at 2 a.m.

`pki-tool check-expiry` checks one certificate at a time. That is fine for a cron job with five certificates. It is not fine for fifty, and it cannot check live TLS endpoints at all. `cert-monitor` does all of this in one pass.

22.1 Scanning Files

```
# Individual files
cert-monitor scan --cert ca-cert.pem --cert server-cert.pem \
    --warn-days 30

# All PEM files in a directory
cert-monitor scan --dir /etc/ssl/certs --warn-days 90

# The pki-tool serial database
cert-monitor scan --db ./server-ca --warn-days 60
```

The `--cert` flag is repeatable. `--dir` scans every `*.pem` file in the given directory. `--db` reads the `serial.db` file created by `pki-tool` and checks every valid certificate recorded in it.

22.2 Scanning from a Config File

For large deployments, maintain a config file with one target per line:

```
# /etc/cert-monitor.conf
/etc/ssl/certs/server.pem
/opt/pki/certs/api/cert.pem
db.internal:5432
api.example.com:443
```

```
cert-monitor scan --config /etc/cert-monitor.conf --warn-days 30
```

Lines that look like `host:port` are probed as TLS endpoints. Everything else is treated as a file path.

22.3 Probing Live TLS Endpoints

```
cert-monitor probe --target example.com:443 \
    --target db.internal:5432 --warn-days 30
```

The `probe` command connects to each target via TLS, retrieves the peer certificate, and checks its expiry. This tells you what is actually deployed, not what is on disk.

Options:

- `--sni NAME` — override the SNI hostname
- `--timeout N` — connection timeout in seconds (default: 10)
- `--config FILE` — host:port list, one per line

22.4 Continuous Monitoring

```
cert-monitor watch --dir /etc/ssl/certs --warn-days 30 \
    --interval 3600
```

The `watch` command runs the scan in a loop, sleeping for `--interval` seconds between iterations. It accepts the same options as `scan`. This is the simplest possible monitoring daemon—no configuration files, no database, no dependencies. Run it under systemd or in a `screen` session.

22.5 Exit Codes

Exit code 0 means all certificates are valid beyond the warning threshold. Exit code 1 means at least one certificate is expiring soon or already expired. This is designed for scripting:

```
if ! cert-monitor scan --db ./server-ca --warn-days 30; then
    echo "ALERT: certificates expiring" | mail -s "PKI Alert" ops@acme.com
fi
```

TLS Inspection

You deployed certificates to your servers. How do you know what is actually negotiated on the wire?

`openssl s_client` is the traditional answer, but its output is a wall of text with the information you need buried in the middle. Parsing it in a script is fragile. `tls-probe` gives you the same information in clean, structured output.

23.1 Basic Probe

```
tls-probe connect --target example.com:443
```

This connects, completes the TLS handshake, and reports:

- TLS protocol version (e.g., TLSv1.3)
- Cipher suite (e.g., TLS_AES_256_GCM_SHA384)
- Leaf certificate details: subject, issuer, serial, dates, key algorithm, SANs
- Full certificate chain (each certificate's subject and issuer)
- Chain verification result (OK or the specific OpenSSL error)

23.2 Custom Trust Store

By default, `tls-probe` uses the system trust store. For private PKI, you need to verify against your own CA:

```
tls-probe connect --target api.internal:443 \
    --trust ./root/ca-cert.pem
```

23.3 SNI Override

When the target IP hosts multiple certificates, specify the hostname the server should select:

```
tls-probe connect --target 10.0.0.1:443 --sni api.example.com
```

23.4 Mutual TLS Testing

To verify that mTLS is working, provide client credentials:

```
tls-probe connect --target api.internal:443 \
    --client-cert ./client/cert.pem \
    --client-key ./client/key.pem \
```

```
    --trust ./root/ca-cert.pem
```

If the server requires client authentication and the provided certificate is valid, the handshake succeeds. If not, you get the specific TLS error.

23.5 Batch Scanning

```
# targets.txt: one HOST:PORT per line
tls-probe scan --config targets.txt --trust ca-cert.pem
```

The `scan` command probes every target in the file and reports results for each. In JSON mode, this is an array of probe results—useful for feeding into a dashboard or alerting system.

23.6 JSON Output

```
tls-probe --output-format json connect --target example.com:443
```

The JSON output includes nested objects for the leaf certificate and chain, making it straightforward to extract specific fields with `jq`:

```
# Extract cipher suite
tls-probe --output-format json connect --target example.com:443 \
    | jq -r .cipher

# Check if chain verifies
tls-probe --output-format json connect --target api.internal:443 \
    --trust ca-cert.pem | jq -r .verify_result
```

Format Conversion

PEM for Linux. DER for embedded systems. PKCS#12 for anything that needs a key and certificate in one file. At some point, you need to convert between them.

`pki-tool create-p12` handles the common case of bundling PEM components into a P12. `keystore-tool` handles everything else: the reverse direction, DER conversion, format inspection, and auto-detection.

24.1 Converting Formats

```
# PEM to DER
keystore-tool convert --in cert.pem --out cert.der --out-format der

# DER to PEM
keystore-tool convert --in cert.der --out cert.pem --out-format pem

# PEM to P12
keystore-tool convert --in cert.pem --out cert.p12 --out-format p12 \
    --pass "secret"
```

The input format is auto-detected by default. You can force it with `--in-format pem|der|p12`.

24.2 Splitting a PKCS#12

```
keystore-tool split --in bundle.p12 --out-dir ./extracted \
    --pass "secret"
```

This extracts the P12 into three PEM files:

- `cert.pem` — the end-entity certificate
- `key.pem` — the private key (mode 0600)
- `chain.pem` — the CA chain (if present)

This is the reverse of `pki-tool create-p12`. You get a P12 from someone, you split it, and you have the components your TLS stack needs.

24.3 Merging PEM Files

```
keystore-tool merge --cert cert.pem --key key.pem \
    --chain chain.pem --out bundle.p12 \
    --pass "secret" --friendly "My Server"
```

The `--friendly` flag sets the PKCS#12 friendly name, which some applications use to identify the entry.

24.4 Inspecting Any Format

```
keystore-tool info --in bundle.p12 --pass "secret"
keystore-tool info --in cert.pem
keystore-tool info --in cert.der
```

The `info` command auto-detects the format and displays the contents. For P12 files, it shows the certificate, chain, and whether a key is present. For PEM and DER, it shows the certificate details.

24.5 Format Auto-Detection

`keystore-tool` identifies formats using a two-step process:

1. **Extension:** `.pem` → PEM, `.der` → DER, `.p12`/`.pfx` → PKCS#12
2. **Content sniffing:** If the extension is ambiguous (`.crt`, `.cer`), the tool checks for a PEM `---BEGIN` header. If absent, it tries PKCS#12 parsing. If that fails, it assumes DER.

This means you can pass files with arbitrary extensions and the tool will usually do the right thing.

Policy Validation

A certificate can be cryptographically valid and operationally wrong. RSA-1024 keys in 2026. Three-year validity when your policy says one year. Missing Subject Alternative Names. Self-signed certificates in a production environment where everything should chain to your CA.

`cert-lint` validates certificates against a written policy. The policy is a text file. The output is a per-rule pass/fail report.

25.1 Writing a Policy

```
# strict.policy
min_key_bits           = 256
max_validity_days      = 397
required_extensions    = subjectKeyIdentifier,authorityKeyIdentifier
allowed_algorithms     = p256,p384,ed25519
required_san           = true
ca_must_have_pathlen   = true
disallow_self_signed   = true
```

One rule per line. Blank lines and `#` comments are ignored.

Rule	Meaning
`min_key_bits`	Minimum key size in bits. EC P-256 = 256, P-384 = 384, RSA-2048 = 2048.
`max_validity_days`	Maximum certificate lifetime. 397 is the current CA/Browser Forum limit for public TLS.
`required_extensions`	Comma-separated list of extensions that must be present.
`allowed_algorithms`	Comma-separated list of permitted key algorithms (p256, p384, ed25519, rsa2048, rsa4096).
`required_san`	If `true`, end-entity certificates must have at least one SAN.
`ca_must_have_pathlen`	If `true`, CA certificates must have a `pathlen` constraint.
`disallow_self_signed`	If `true`, self-signed certificates fail the lint.

Table 25.1: Policy rules

25.2 Linting a Certificate

```
cert-lint lint --cert server-cert.pem --policy strict.policy
```

Output is one line per rule, each marked PASS or FAIL with a description of what was checked and what was found. Exit code 0 if all rules pass, 1 if any rule fails.

25.3 Linting a Directory

```
cert-lint lint --dir /etc/ssl/certs --policy strict.policy
```

This lints every `*.pem` file in the directory. Each certificate is checked independently.

25.4 Displaying a Policy

```
cert-lint policy --file strict.policy
```

This parses and displays the policy file, confirming that the rules are valid and showing the parsed values. Useful for verifying a policy before deploying it.

25.5 Practical Use

The value of `cert-lint` is not in catching deliberate violations. It is in catching drift. Six months after you wrote the policy, someone issues a certificate with the wrong algorithm because they copied an old script. The linter catches it.

Run it in CI against every certificate your build pipeline produces. Run it periodically against your deployed certificates. Make the policy file part of your repository, reviewed alongside code changes.

Digital Signatures

Certificates prove identity. Signatures prove integrity. If you issue certificates, you probably also have artifacts that need signing: firmware images, release tarballs, configuration files, audit logs.

`sign-tool` creates and verifies detached CMS/PKCS#7 digital signatures using the same X.509 certificates you already have.

26.1 Signing a File

```
sign-tool sign --key signer-key.pem --cert signer-cert.pem \
    --in firmware.bin --out firmware.bin.sig
```

This produces a detached signature in DER format. The original file is not modified. The signature contains the signer's certificate and the digest, signed with the signer's private key.

Options:

- `--hash sha256|sha384|sha512` — digest algorithm (default: SHA-256)
- `--format der|pem` — signature output format (default: DER)
- `--chain FILE` — include the certificate chain in the signature

26.2 Verifying a Signature

```
sign-tool verify --cert signer-cert.pem --ca-cert ca-cert.pem \
    --in firmware.bin --sig firmware.bin.sig
```

Verification checks three things:

1. The signature is mathematically valid against the file content
2. The signer's certificate matches
3. The signer's certificate chains to the provided CA certificate

If any check fails, the tool reports the specific failure and exits with code 1.

26.3 Tamper Detection

If the file has been modified after signing, verification fails:

```
echo "tampered" >> firmware.bin
sign-tool verify --cert signer-cert.pem --ca-cert ca-cert.pem \
    --in firmware.bin --sig firmware.bin.sig
# Result: FAIL - verification error
```

This is the point. The signature binds the signer's identity to the exact contents of the file at the time of signing.

26.4 Hash Algorithms

SHA-256 is the default and the right choice for most purposes. SHA-384 and SHA-512 are available for environments that require them (some government and defense standards mandate SHA-384).

```
sign-tool sign --key key.pem --cert cert.pem \
    --in firmware.bin --out firmware.bin.sig --hash sha384
```

The verification side auto-detects the hash algorithm from the signature, so the verifier does not need to know which hash was used.

Hybrid Encryption

Sometimes you need to encrypt a file for specific people. Not a TLS stream, not a VPN tunnel—a file. A configuration blob with secrets. A database backup. A license key.

`secret-tool` uses CMS EnvelopedData, which is the standard way to encrypt data using X.509 certificates. It generates a random symmetric key (AES), encrypts the data with it, then encrypts the symmetric key once per recipient using their public key. Each recipient can decrypt using their private key.

27.1 Encrypting

```
# For a single recipient
secret-tool encrypt --cert alice-cert.pem \
    --in secret.txt --out secret.cms

# For multiple recipients
secret-tool encrypt --cert alice-cert.pem --cert bob-cert.pem \
    --in secret.txt --out secret.cms
```

Each `--cert` flag adds a recipient. The encrypted output can be decrypted by any of the listed recipients.

Options:

- `--cipher aes-128-cbc|aes-256-cbc` — symmetric cipher (default: AES-256-CBC)
- `--format der|pem` — output format (default: DER)

27.2 Decrypting

```
secret-tool decrypt --key alice-key.pem \
    --in secret.cms --out secret.txt
```

The recipient provides their private key. The tool identifies the matching recipient entry in the CMS structure and decrypts. If the key does not match any recipient, decryption fails.

27.3 Why CMS, Not GPG

GPG uses its own key format, its own trust model, and its own tooling ecosystem. If you already have X.509 certificates (which you do, since you are reading this book), CMS lets you reuse them for encryption without maintaining a parallel key infrastructure.

The trade-off: CMS is less widely known, and the encrypted files are not compatible with GPG. If your recipients use GPG, use GPG. If your recipients are systems with X.509 certificates, use `secret-tool`.

27.4 Binary Files

CMS encryption handles binary data without issues:

```
secret-tool encrypt --cert admin-cert.pem \
    --in database-backup.tar.gz --out database-backup.tar.gz.cms
```

The output is the same size as the input plus a few hundred bytes of CMS overhead. There is no base64 expansion (unless you use `--format pem`, which adds base64 encoding).

JSON Output for Automation

Every command in every tool supports `--output-format json`. This is designed for three use cases:

28.1 CI/CD Pipelines

Parse the output of any command in your deployment scripts:

```
DAYS=$(pki-tool --output-format json check-expiry \
    --cert ./cert.pem | jq .days_remaining)
if [ "$DAYS" -lt 30 ]; then
    echo "Certificate expiring in $DAYS days"
fi
```

```
# Check TLS configuration in CI
CIPHER=$(tls-probe --output-format json connect \
    --target api.staging:443 | jq -r .cipher)
if [ "$CIPHER" != "TLS_AES_256_GCM_SHA384" ]; then
    echo "Unexpected cipher: $CIPHER"
    exit 1
fi
```

28.2 Inventory Management

Dump the serial database as JSON and feed it into your CMDB or monitoring system:

```
pki-tool --output-format json list --ca ./server-ca > certs.json
```

28.3 Audit Logging

Log every certificate operation as a structured JSON event:

```
pki-tool --output-format json issue-cert --ca ./ca \
    --cn "server" --out ./srv --db >> /var/log/pki-audit.json

cert-lint --output-format json lint --cert ./srv/cert.pem \
    --policy strict.policy >> /var/log/pki-audit.json
```

Custom Scripts

For recurring certificate generation tasks, write a shell script. The `custom/` directory in the repository contains an example: `generate-roip-certs.sh`.

This script generates a complete PKI hierarchy for a specific project (a Radio-over-IP system), producing exactly four PKCS#12 files. It is meant as a template for your own project-specific scripts.

The pattern is straightforward:

1. Accept an output directory and optional parameters
2. Find the tool binaries
3. Create a temporary workspace for intermediate PEM files
4. Run `pki-tool` commands to build the hierarchy
5. Bundle the final PKCS#12 files into the output directory
6. Clean up the temporary workspace

```
#!/bin/bash
set -euo pipefail
WORKDIR=$(mktemp -d)
trap "rm -rf $WORKDIR" EXIT

pki-tool init-ca --cn "My Root CA" --days 3650 --out "$WORKDIR/root"
pki-tool create-sub-ca --parent "$WORKDIR/root" \
    --cn "My Server CA" --days 1825 --out "$WORKDIR/srv-ca"
pki-tool issue-cert --ca "$WORKDIR/srv-ca" --cn "server" \
    --days 365 --out "$WORKDIR/srv"

pki-tool create-p12 --cert "$WORKDIR/srv/cert.pem" \
    --key "$WORKDIR/srv/key.pem" \
    --chain "$WORKDIR/srv-ca/chain.pem" --out "$1/server.p12"
pki-tool create-p12 --cert "$WORKDIR/root/ca-cert.pem" \
    --key "$WORKDIR/root/ca-key.pem" --out "$1/trust.p12"
```

The intermediate PEM files are temporary artifacts. Only the PKCS#12 files matter. Clean up the rest.

Reference Card

For quick reference, here is every command across all seven tools.

30.1 pki-tool

Command	Key Options
init-ca	--cn --days --out [--algo --pathlen --encrypt-key --db --permitted-dns --excluded-dns]
create-sub-ca	--parent --cn --out [--days --algo --pathlen --encrypt-key --db --permitted-dns --excluded-dns]
issue-cert	--ca --cn --out [--days --algo --san --profile --encrypt-key --db]
create-p12	--cert --key --out [--chain --pass --pass-file]
verify	--cert --ca-cert
inspect	--cert | --p12 | --csr [--pass --pass-file]
check-expiry	--cert [--warn-days]
create-csr	--cn --out [--algo --san --encrypt-key]
sign-csr	--ca --csr --out [--days --san --profile --db]
revoke	--ca (--cert | --serial)
generate-crl	--ca [--days --out]
renew	--cert --ca --out [--days --algo --san --encrypt-key --db]
list	--ca

Table 30.1: pki-tool command reference

30.2 cert-monitor

Command	Key Options
scan	--cert FILE (repeatable), --dir DIR, --db CA_DIR, --config FILE [--warn-days]
watch	Same as scan plus --interval N
probe	--target HOST:PORT (repeatable), --config FILE [--sni --warn-days --timeout]

Table 30.2: cert-monitor command reference

Command	Key Options
`connect`	`--target HOST:PORT` [`--sni --trust --client-cert --client-key --timeout`]
`scan`	`--config FILE` [`--trust --sni --client-cert --client-key --timeout`]

Table 30.3: `tls-probe` command reference

Command	Key Options
`convert`	`--in FILE` [`--in-format`] `--out FILE --out-format pem|der|p12` [`--pass --pass-file --out-pass`]
`split`	`--in FILE --out-dir DIR` [`--pass --pass-file`]
`merge`	`--cert F --key F --out FILE` [`--chain F --out-format --pass --pass-file --friendly NAME`]
`info`	`--in FILE` [`--pass --pass-file`]

Table 30.4: `keystore-tool` command reference

30.3 tls-probe

30.4 keystore-tool

30.5 cert-lint

30.6 sign-tool

30.7 secret-tool

Command	Key Options
`lint`	(`--cert FILE` | `--dir DIR`) `--policy FILE`
`policy`	`--file FILE`

Table 30.5: `cert-lint` command reference

Command	Key Options
`sign`	`--key F --cert F --in FILE --out FILE` [`--chain F --hash ALG --format FMT`]
`verify`	`--cert F --ca-cert F --in FILE --sig FILE` [`--format FMT`]

Table 30.6: `sign-tool` command reference

Command	Key Options
`encrypt`	`--cert F` (repeatable) `--in FILE --out FILE` [`--cipher ALG --format FMT`]
`decrypt`	`--key F --in FILE --out FILE` [`--cert F --pass P --pass-file F --format FMT`]

Table 30.7: `secret-tool` command reference

Common Mistakes and Troubleshooting

Here are the errors people actually make, and how to fix them.

31.1 "Cannot read CA key"

This means the CA private key file could not be opened or parsed. Three possible causes:

1. **Wrong directory.** You passed `--ca ./ca` but the key is in `./root-ca`. Check that `<dir>/ca-key.pem` exists.
2. **Encrypted key, no passphrase.** The key was generated with `--encrypt-key` but `PKI_KEYSTORE_PASS` is not set. Export it: `export PKI_KEYSTORE_PASS="your-pass"`
3. **File permissions.** The key has 0600 permissions owned by a different user. You need to run as the same user who created the CA, or fix the permissions.

31.2 Forgot `--db` During Issuance

You issued certificates without `--db` and now you cannot revoke them or list them. The serial database does not exist.

The fix: inspect each certificate to extract its serial number and CN, and manually create the database file:

```
# Get the serial from the certificate
pki-tool --output-format json inspect --cert ./cert.pem | \
    jq -r '[.serial, .subject, .not_after, "V"] | join("|")'
```

Append the output to `<ca-dir>/serial.db`. Format: `SERIAL|CN|NOT_AFTER|STATUS`

Better: always use `--db` from the start. It costs nothing and saves you from this situation.

31.3 Chain Verification Fails

`pki-tool verify` returns "FAIL" even though the certificate looks correct. Check these in order:

1. Are you verifying against the right CA? Use the `chain.pem` from the intermediate CA, not the root CA cert alone (unless the certificate was signed directly by the root).
2. Has the certificate expired? Run `check-expiry` to see.
3. Has the CA certificate expired? CAs expire too.
4. Did you use name constraints? A certificate for `api.other.com` will fail verification against a CA that only permits `.acme.com`.

31.4 Wrong Profile After Issuance

You issued a certificate with `tls-server` but need `tls-mutual`. Certificates are immutable once signed—you cannot change extensions after the fact. Re-issue:

```
pki-tool issue-cert --ca ./ca --cn "same-cn" \
    --profile tls-mutual --san "DNS:same-san" --out ./new-dir --db
```

If the old certificate was tracked with `--db`, revoke it.

31.5 PKCS#12 Import Fails on Another System

Some older systems (Java keystores, Windows) struggle with modern PKCS#12 encryption algorithms. This is an OpenSSL default, not a `pki-tools` issue. If you hit this, convert the P12 with legacy algorithms:

```
openssl pkcs12 -in modern.p12 -out temp.pem -nodes \
    -passin pass:your-pass
openssl pkcs12 -export -in temp.pem -out legacy.p12 \
    -passout pass:your-pass -legacy
rm temp.pem
```

Alternatively, use `keystore-tool` to split the P12 into PEM components and rebuild it on the target system.

31.6 Ed25519 Not Recognized by Client

Ed25519 is not universally supported. Java 15+ supports it. OpenSSL 1.1.1+ supports it. Older Go versions, some embedded TLS stacks, and older Windows builds do not. If in doubt, use P-256—it works everywhere.

31.7 `sign-tool verify` Fails with "unsuitable certificate purpose"

This happens when the signer certificate has a restrictive `extKeyUsage` that does not match CMS signing expectations. `sign-tool` uses `X509_PURPOSE_ANY` internally to avoid this, but if you are verifying with a different tool, you may hit it. The fix is to use a certificate with a signing-compatible profile (`code-signing` or `tls-mutual`).

31.8 `cert-lint` Reports Failures on Certificates You Just Issued

Read the failure messages. Common causes:

- `required_san = true` but you did not pass `--san` when issuing.
- `allowed_algorithms = p256,p384` but the certificate uses RSA. Update the policy or re-issue with an allowed algorithm.

- `disallow_self_signed = true` and you are linting the root CA certificate. Root CAs are self-signed by definition. Either exclude them from the lint or drop that rule for CA certificates.

Putting It All Together

Here is a realistic workflow for a small team managing internal services, using the full tool suite.

32.1 Initial Setup (Once)

```
# Generate a strong passphrase
export PKI_KEYSTORE_PASS=$(openssl rand -hex 16)
echo "PKI_KEYSTORE_PASS=$PKI_KEYSTORE_PASS" >> ~/.pki-env

# Root CA (10 years, RSA-4096, path length 1)
pki-tool init-ca --cn "Acme Internal Root CA" --days 3650 \
    --algo rsa4096 --pathlen 1 --encrypt-key --out /opt/pki/root --db

# Server CA (5 years, name-constrained)
pki-tool create-sub-ca --parent /opt/pki/root \
    --cn "Acme Server CA" --days 1825 \
    --permitted-dns ".acme.internal" \
    --encrypt-key --out /opt/pki/server-ca --db

# Client CA (5 years)
pki-tool create-sub-ca --parent /opt/pki/root \
    --cn "Acme Client CA" --days 1825 \
    --encrypt-key --out /opt/pki/client-ca --db
```

Store `/opt/pki/root/ca-key.pem` offline. The root key is needed only to create or replace intermediate CAs.

Write the organizational policy:

```
# /opt/pki/policy.txt
min_key_bits           = 256
max_validity_days      = 397
required_extensions    = subjectKeyIdentifier,authorityKeyIdentifier
allowed_algorithms     = p256,p384
required_san           = true
```

32.2 Issuing Certificates (As Needed)

```
source ~/.pki-env

pki-tool issue-cert --ca /opt/pki/server-ca \
    --cn "api.acme.internal" --days 365 \
    --san "DNS:api.acme.internal" \
    --profile tls-server \
```

```
    --out /opt/pki/certs/api --db

# Validate against policy before deployment
cert-lint lint --cert /opt/pki/certs/api/cert.pem \
    --policy /opt/pki/policy.txt

# Bundle for deployment
pki-tool create-p12 --cert /opt/pki/certs/api/cert.pem \
    --key /opt/pki/certs/api/key.pem \
    --chain /opt/pki/server-ca/chain.pem \
    --out /opt/pki/bundles/api.p12
```

32.3 Monitoring (Weekly Cron)

```
#!/bin/bash
# Scan the serial database for expiring certs
if ! cert-monitor scan --db /opt/pki/server-ca --warn-days 30 \
     >/dev/null 2>&1; then
    cert-monitor --output-format json scan \
        --db /opt/pki/server-ca --warn-days 30 \
        | mail -s "PKI: Certificates Expiring" ops@acme.com
fi

# Verify live TLS endpoints match expectations
tls-probe --output-format json scan \
    --config /opt/pki/endpoints.conf \
    --trust /opt/pki/root/ca-cert.pem \
    >> /var/log/tls-audit.json
```

32.4 Renewal (When Warned)

```
source ~/.pki-env

pki-tool renew --cert /opt/pki/certs/api/cert.pem \
    --ca /opt/pki/server-ca --days 365 \
    --out /opt/pki/certs/api-v2 --db

# Validate the renewed cert
cert-lint lint --cert /opt/pki/certs/api-v2/cert.pem \
    --policy /opt/pki/policy.txt

# Re-bundle
pki-tool create-p12 --cert /opt/pki/certs/api-v2/cert.pem \
    --key /opt/pki/certs/api-v2/key.pem \
    --chain /opt/pki/server-ca/chain.pem \
    --out /opt/pki/bundles/api.p12

# Deploy the new P12, then revoke the old cert
pki-tool revoke --ca /opt/pki/server-ca \
    --cert /opt/pki/certs/api/cert.pem
pki-tool generate-crl --ca /opt/pki/server-ca --days 7
```

32.5 Signing a Release (As Needed)

```
# Sign the release artifact
sign-tool sign --key /opt/pki/certs/build/key.pem \
    --cert /opt/pki/certs/build/cert.pem \
    --chain /opt/pki/server-ca/chain.pem \
    --in release-v2.1.tar.gz --out release-v2.1.tar.gz.sig

# Verify (on the receiving end)
sign-tool verify --cert /opt/pki/certs/build/cert.pem \
    --ca-cert /opt/pki/root/ca-cert.pem \
    --in release-v2.1.tar.gz --sig release-v2.1.tar.gz.sig
```

32.6 Encrypting Secrets (As Needed)

```
# Encrypt a config file for the ops team
secret-tool encrypt --cert /opt/pki/certs/alice/cert.pem \
    --cert /opt/pki/certs/bob/cert.pem \
    --in production.env --out production.env.cms

# Alice decrypts on her machine
secret-tool decrypt --key ~/pki/alice-key.pem \
    --in production.env.cms --out production.env
```

32.7 Quarterly Audit

```
# Dump certificate inventory
pki-tool --output-format json list --ca /opt/pki/server-ca
pki-tool --output-format json list --ca /opt/pki/client-ca

# Lint all deployed certificates against policy
cert-lint --output-format json lint \
    --dir /opt/pki/certs --policy /opt/pki/policy.txt

# Probe all endpoints
tls-probe --output-format json scan \
    --config /opt/pki/endpoints.conf \
    --trust /opt/pki/root/ca-cert.pem
```

Feed the JSON into your compliance reporting system.

That is the whole story. Generate keys, issue certificates, validate them against policy, bundle them for deployment, monitor their expiry, probe your live endpoints, sign your artifacts, encrypt your secrets, revoke what needs revoking, and renew before things expire. Seven tools, each doing one thing. The cryptographic details are handled correctly so that you can focus on the operational side—which, in practice, is where all the real problems live.

Part IV

Getting pki-tools

Download the Trial

A trial edition is available for evaluation in development environments. It includes all seven tools with a shared limit of 100 combined invocations.

33.1 Download

Download the latest trial archive from the project's release page or request it by email:

- **File:** `pki-tools-trial-linux-x86_64.tar.gz`
- **Platform:** Linux x86_64
- **Requires:** OpenSSL 3.x shared libraries (`libssl.so.3`, `libcrypto.so.3`)
- **Contact:** chiradip@chiradip.com

33.2 Install

Extract the archive and either place the binaries on your `PATH` or run them directly from the extracted directory:

```
tar xzf pki-tools-trial-linux-x86_64.tar.gz

# Option A: install system-wide
sudo cp pki-tool cert-monitor tls-probe keystore-tool \
        cert-lint sign-tool secret-tool /usr/local/bin/

# Option B: run in place
./pki-tool init-ca --cn "My Root CA" --days 3650 --out ./my-ca
```

33.3 Trial Behavior

All seven tools share a single usage counter stored at `~/.pki-tools/.usage.dat`.

- **Invocations 1–89**: tools run normally, no messages.
- **Invocations 90–100**: tools run normally; a warning is printed to stderr:

  ```
  Trial warning: 93/100 invocations used.
  ```

- **After 100**: all tools refuse to run:

  ```
  Trial expired: 100/100 invocations used.
  Please purchase a license.
  ```

33.4 Restrictions

The trial edition is licensed for use in **development environments only**. Deployment to test, QA, staging, pre-production, production, or any other non-development environment is prohibited. Please refer to the accompanying `readme-trial-howto.md` for the full End-User License Agreement.

Buy a Full License

A full license removes all invocation limits and permits deployment to every environment—development, test, staging, and production.

34.1 License Models

Annual Subscription
: A yearly license with access to all updates and patches released during the subscription period. Renews annually.

Perpetual License
: A one-time purchase granting unlimited use of the current version. No recurring fees. Future major versions require a separate purchase.

34.2 Pricing

Licenses are sold in blocks of 10 users. The first block covers up to 10 users; each additional block of 1–10 users is added at the incremental rate.

Users	Annual Subscription	Perpetual License
1 – 10	$7,000	$10,000
+10 users	+$5,000	+$7,000

Examples:

Team Size	Annual	Perpetual
5 users	$7,000	$10,000
10 users	$7,000	$10,000
15 users	$12,000	$17,000
25 users	$17,000	$24,000
50 users	$32,000	$45,000

All prices are in USD.

34.3 Buy Online

Purchase directly through our payment portal:

First 1–10 users (required):

- Annual Subscription — $7,000
- Perpetual License — $10,000

Each additional block of 1–10 users:

- Annual Subscription Add-On — $5,000
- Perpetual License Add-On — $7,000

Purchase the first block, then add as many additional blocks as needed to cover your team. For example, a 25-user perpetual license requires one primary block plus two add-on blocks.

34.4 Volume Discounts

If your organization has a large number of users, you may qualify for a volume discount. Contact us to discuss pricing tailored to your needs.

34.5 Contact

- **Email:** chiradip@chiradip.com

We will work with you to determine the right license tier, generate your licensed build, and get your team up and running.